Networks as NetworkX graphs

Lab handout 2

October 2026

To be read while running the script labs/lab1_networkx_tour.py.

Introduction

All the operations that PyNet2Plan performs on a network are based on NetworkX, the Python library for graphs that is also used in the rest of the course. There is no special class for networks: a network is a `networkx.DiGraph`, the traffic is a list of demands, and an algorithm is a Python function that receives both.

The script `labs/lab1_networkx_tour.py` illustrates the main ideas in seven short cells. It is run from the lab folder, with the virtual environment active:

```
python labs/lab1_networkx_tour.py           # prints the results and saves the figures in
results/networkx_tour/
python labs/lab1_networkx_tour.py --show    # also opens the figures
```

The cells, separated by `# %%` lines, can also be executed one at a time in VS Code. In the following sections each cell is reproduced together with the output it prints.

1. A network is a directed graph

A link is unidirectional: a pair of fibres between two cities corresponds to two links, one in each direction, and each of them has its own capacity. In NetworkX such a network is a directed graph (`DiGraph`) whose nodes and edges carry attributes. Nodes have a drawing position `x`, `y` and, optionally, the geographic coordinates `lat` and `lon`. Links have a number of `fibers` (1 by default), a `capacity`, which is the number of wavelengths of each fiber, and a `length` (km). A link therefore offers `fibers × capacity` wavelengths. All the networks used in the lab have one fiber per link.

```
section("1. A network is a networkx.DiGraph")
H = nx.DiGraph(name="Tiny")
H.add_node("A", x=0, y=0)
H.add_node("B", x=100, y=0)
H.add_node("C", x=50, y=80)
H.add_edge("A", "B", capacity=40, fibers=1, length=100.0)   # one direction ...
H.add_edge("B", "A", capacity=40, fibers=1, length=100.0)   # ... and the opposite one
H.add_edge("B", "C", capacity=10, fibers=2, length=94.3)    # 2 fibers of 10 wavelengths each
print("nodes:", list(H.nodes(data=True)))
print("links:", list(H.edges(data=True)))
bc = H.edges["B", "C"]
print("capacity of B->C:", bc["fibers"], "fibers x", bc["capacity"], "wavelengths =",
      bc["fibers"] * bc["capacity"], "wavelengths")
# the PyNet2Plan helpers build exactly the same graph, checking the values for you:
H2 = network.new_network("Tiny")
for n, x, y in [("A", 0, 0), ("B", 100, 0), ("C", 50, 80)]:
    network.add_node(H2, n, x=x, y=y)
network.add_link(H2, "A", "B", capacity=40, bidirectional=True)   # length from the positions
network.add_link(H2, "B", "C", capacity=10, fibers=2)          # fibers=1 if not given
print("same links with the helpers:", sorted(H2.edges) == sorted(H.edges))
```

```
nodes: [('A', {'x': 0, 'y': 0}), ('B', {'x': 100, 'y': 0}), ('C', {'x': 50, 'y': 80})]
links: [('A', 'B', {'capacity': 40, 'fibers': 1, 'length': 100.0}), ('B', 'A', {'capacity': 40,
'fibers': 1, 'length': 100.0}), ('B', 'C', {'capacity': 10, 'fibers': 2, 'length': 94.3})]
capacity of B->C: 2 fibers x 10 wavelengths = 20 wavelengths
same links with the helpers: True
```

The functions of `pynet2plan.network` (`add_node`, `add_link`, `load_topology`, `save_topology` and so on) build exactly the same graph. In addition they check the values (for example, a capacity must be a whole number and a link cannot start and end at the same node) and compute the lengths from the positions of the nodes.

2. The Spain-7 network

```
section("2. The Spain-7 network of the lab")
G = network.load_topology(os.path.join(ROOT, "data", "topologies", "spain7.json"))
print(type(G).__name__, "-", G.graph["name"], "-", G.number_of_nodes(), "nodes,",
G.number_of_edges(), "links")
print("Madrid:", G.nodes["Madrid"])
print("links leaving Madrid:", list(G.out_edges("Madrid")))
print("links entering Madrid:", list(G.in_edges("Madrid")))
print("out-degree:", dict(G.out_degree()))
print(f"{'link':22s} {'fibers':>6s} {'capacity':>8s} {'length (km)':>12s}")
for u, v, a in sorted(G.edges(data=True)):
    print(f"{u + ' -> ' + v:22s} {a['fibers']:6d} {a['capacity']:8d} {a['length']:12.1f}")

pos = {n: (a["x"], a["y"]) for n, a in G.nodes(data=True)}     # drawing positions
U = G.to_undirected()                                          # one line per pair of links
fig, ax = plt.subplots(figsize=(7.5, 6.2))
nx.draw_networkx_edges(U, pos, ax=ax, width=2, edge_color="#546e7a")
nx.draw_networkx_nodes(U, pos, ax=ax, node_size=900, node_color=BLUE, edgecolors="white")
nx.draw_networkx_labels(U, pos, ax=ax, font_size=10, font_color="white", font_weight="bold",
                        labels={n: n[:3] for n in U})
nx.draw_networkx_edge_labels(U, pos, ax=ax, font_size=10,
                             edge_labels={(u, v): f"{a['length']:.0f} km" for u, v, a in
U.edges(data=True)})
ax.set_title("Spain-7, link lengths in km")
ax.set_axis_off()
ax.margins(0.08)
save(fig, "nx_spain7.png")
```

```
DiGraph - Spain-7 - 7 nodes, 16 links
Madrid: {'x': -412305.1300001266, 'y': 4926696.669635532, 'lat': 40.4168, 'lon': -3.7038, 'up': True}
links leaving Madrid: [('Madrid', 'Zaragoza'), ('Madrid', 'Sevilla'), ('Madrid', 'Valencia')]
links entering Madrid: [('Valencia', 'Madrid'), ('Sevilla', 'Madrid'), ('Zaragoza', 'Madrid')]
out-degree: {'Madrid': 3, 'Barcelona': 2, 'Valencia': 3, 'Sevilla': 2, 'Zaragoza': 2, 'Malaga': 2,
'Murcia': 2}
link                   fibers capacity  length (km)
Barcelona -> Valencia       1       40        303.1
Barcelona -> Zaragoza       1       40        256.2
Madrid -> Sevilla           1       40        390.2
Madrid -> Valencia          1       40        302.6
Madrid -> Zaragoza          1       40        272.9
Malaga -> Murcia            1       40        323.3
Malaga -> Sevilla           1       40        157.3
Murcia -> Malaga            1       40        323.3
Murcia -> Valencia          1       40        176.9
Sevilla -> Madrid           1       40        390.2
Sevilla -> Malaga           1       40        157.3
Valencia -> Barcelona       1       40        303.1
```

```
Valencia -> Madrid          1       40        302.6
Valencia -> Murcia          1       40        176.9
Zaragoza -> Barcelona       1       40        256.2
Zaragoza -> Madrid          1       40        272.9
   figure saved: results/networkx_tour/nx_spain7.png
```

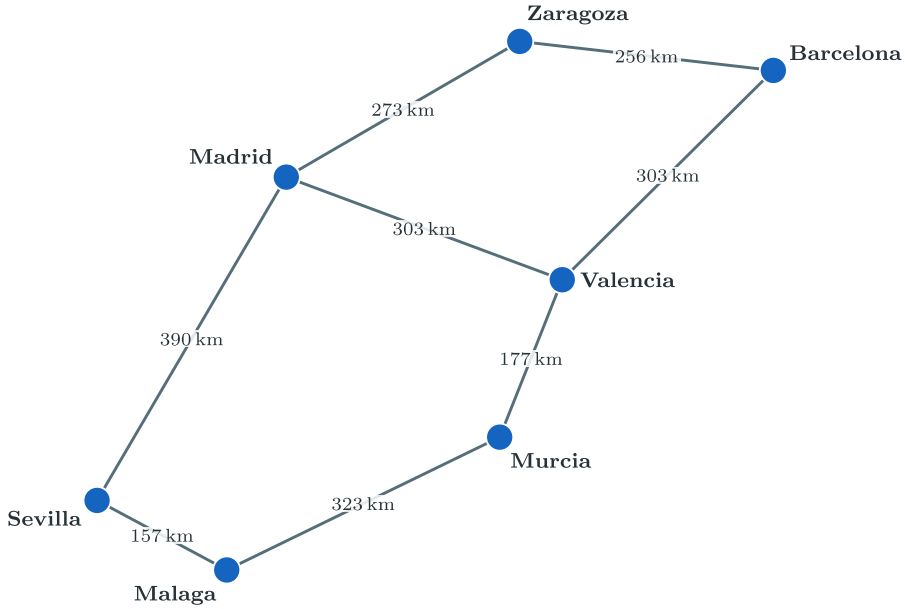


Figure 1. Spain-7 and the length of its links. The cell above draws it with the functions of NetworkX; the method `G.to_undirected()` merges the two directions of each link into a single line. The figures of this handout show the same data redrawn with TikZ.

The table below lists the NetworkX expressions used most often in the lab.

purpose	NetworkX
nodes and links	`G.nodes`, `G.edges`; with their attributes `G.nodes(data=True)`, `G.edges(data=True)`
a single attribute	`G.nodes["Madrid"]["lat"]`, `G.edges["Madrid", "Zaragoza"]["capacity"]`, `G.edges["Madrid", "Zaragoza"]["fibers"]`
links leaving or entering a node n	`G.out_edges(n)`, `G.in_edges(n)`
neighbours of a node	`G.successors(n)`, `G.predecessors(n)`, `G.out_degree(n)`
copies and sub-networks	`G.copy()`, `G.subgraph(nodes)`, `G.edge_subgraph(links)`

3. Shortest paths

The meaning of "shortest" depends on the weight. Without a weight NetworkX counts the hops, that is the number of links; with `weight="length"` it adds the lengths in kilometres. From Sevilla to Valencia the two criteria give different paths.

```
section("3. Shortest paths")
s, t = "Sevilla", "Valencia"
p_hops = nx.shortest_path(G, s, t)                    # fewest links (hops)
p_km = nx.shortest_path(G, s, t, weight="length")     # shortest in km
km = nx.path_weight(G, p_km, weight="length")
print(f"fewest hops : {' > '.join(p_hops)}   ({len(p_hops) - 1} hops, "
      f"{nx.path_weight(G, p_hops, weight='length'):.0f} km)")
print(f"shortest km : {' > '.join(p_km)}   ({len(p_km) - 1} hops, {km:.0f} km)")
hops = dict(nx.shortest_path_length(G))                # hop distance of every pair
print("hop distance Zaragoza -> every node:", hops["Zaragoza"])


def draw_paths(ax, paths, colors, title):
    nx.draw_networkx_edges(U, pos, ax=ax, width=1.5, edge_color="#cfd8dc")
    for p, c in zip(paths, colors):
        nx.draw_networkx_edges(U, pos, ax=ax, edgelist=list(zip(p[:-1], p[1:])), width=5,
edge_color=c)
    nx.draw_networkx_nodes(U, pos, ax=ax, node_size=620, node_color=BLUE, edgecolors="white")
    nx.draw_networkx_labels(U, pos, ax=ax, font_size=8.5, font_color="white", font_weight="bold",
                            labels={n: n[:3] for n in U})
    ax.set_title(title, fontsize=11.5)
    ax.set_axis_off()
    ax.margins(0.08)


fig, axes = plt.subplots(1, 2, figsize=(11, 4.8))
draw_paths(axes[0], [p_hops], [ORANGE], f"{s} to {t}, fewest hops ({len(p_hops) - 1})")
draw_paths(axes[1], [p_km], [AQUA], f"{s} to {t}, shortest length ({km:.0f} km)")
save(fig, "nx_shortest_paths.png")
```

```
fewest hops : Sevilla > Madrid > Valencia   (2 hops, 693 km)
shortest km : Sevilla > Malaga > Murcia > Valencia   (3 hops, 658 km)
hop distance Zaragoza -> every node: {'Zaragoza': 0, 'Madrid': 1, 'Barcelona': 1, 'Sevilla': 2,
'Valencia': 2, 'Malaga': 3, 'Murcia': 3}
   figure saved: results/networkx_tour/nx_shortest_paths.png
```

(a) fewest hops: 2 hops

(b) shortest length: 658 km



Figure 2. Path with the fewest hops (left) and shortest path in kilometres (right) from Sevilla to Valencia.

4. The k shortest paths

The function `nx.shortest_simple_paths` enumerates the loop-free paths between two nodes in order of increasing length. The first k of them are the candidate paths of the route formulations and of the k-shortest-path heuristic studied in the next labs; the helper `pynet2plan.helpers.k_shortest_paths(G, s, t, k)` is implemented in this way.

```
section("4. The k shortest paths: candidate routes")
s, t = "Zaragoza", "Malaga"
k = 3
paths = list(islice(nx.shortest_simple_paths(G, s, t, weight="length"), k))
for i, p in enumerate(paths, 1):
    print(f"  path {i}: {' > '.join(p):45s} {len(p) - 1} hops, "
          f"{nx.path_weight(G, p, weight='length'):6.0f} km")
fig, axes = plt.subplots(1, k, figsize=(14, 4.6))
for i, (ax, p, c) in enumerate(zip(axes, paths, [BLUE, ORANGE, AQUA]), 1):
    draw_paths(ax, [p], [c], f"path {i}: {len(p) - 1} hops, {nx.path_weight(G, p,
weight='length'):.0f} km")
fig.suptitle(f"The {k} shortest paths from {s} to {t}", fontsize=13)
save(fig, "nx_k_shortest_paths.png")
```

```
  path 1: Zaragoza > Madrid > Sevilla > Malaga          3 hops,    820 km
  path 2: Zaragoza > Barcelona > Valencia > Murcia > Malaga 4 hops,   1059 km
  path 3: Zaragoza > Madrid > Valencia > Murcia > Malaga 4 hops,   1076 km
   figure saved: results/networkx_tour/nx_k_shortest_paths.png
```

(a) path 1: 3 hops, 820 km (b) path 2: 4 hops, 1059 km (c) path 3: 4 hops, 1076 km

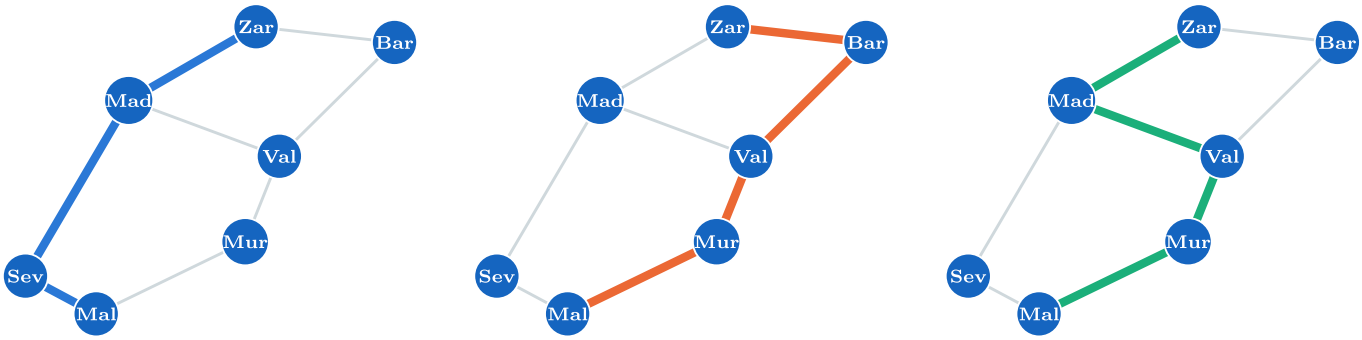


Figure 3. The three shortest paths from Zaragoza to Malaga by length.

5. Connectivity, failures and cuts

NetworkX also answers questions on the robustness of a network: whether it is connected, which single failures disconnect it, and so on. A failure is simply the removal of an edge. In the graphical interface, *Fail / repair selected* has the same effect on the copy of the network that is given to the algorithms.

```
section("5. Connectivity, failures and cuts")
print("strongly connected:", nx.is_strongly_connected(G))
print("links that disconnect the network if they fail (bridges):", list(nx.bridges(U)) or "none")
print("node connectivity:", nx.node_connectivity(U), "  edge connectivity:", nx.edge_connectivity(U))
F = G.copy()
F.remove_edges_from([("Madrid", "Valencia"), ("Valencia", "Madrid")])     # a fiber cut
print("Madrid -> Murcia before the cut:", " > ".join(nx.shortest_path(G, "Madrid", "Murcia")))
print("Madrid -> Murcia after the cut :", " > ".join(nx.shortest_path(F, "Madrid", "Murcia")))

# Cut condition: the traffic that must leave a set of nodes S cannot exceed the
# capacity of the links leaving S. With one fiber and the same capacity C on every link:
#     C >= traffic(S -> outside) / number of links (S -> outside)     for every S
D = traffic.load_traffic(os.path.join(ROOT, "data", "traffic",
"spain7_traffic_split_vs_unsplit.csv"), G)
best = None
for size in range(1, G.number_of_nodes()):
    for S in itertools.combinations(G.nodes, size):
        S = set(S)
        links = len(list(nx.edge_boundary(G, S)))                # links from S to the rest
        need = sum(d.traffic for d in D if d.origin in S and d.destination not in S)
        if links and (best is None or math.ceil(need / links) > best[0]):
            best = (math.ceil(need / links), need, links, S)
C_min, need, links, S = best
print(f"tightest cut: traffic into {sorted(set(G) - S)} = {need} λ over {links} links"
      f"  ->  every link needs C >= {C_min}")

fig, ax = plt.subplots(figsize=(7.5, 6.2))
inside = set(G) - S
cut = [(u, v) for u, v in U.edges if (u in S) != (v in S)]          # drawn once per pair
nx.draw_networkx_edges(U, pos, ax=ax, width=2, edge_color="#b0bec5")
nx.draw_networkx_edges(U, pos, ax=ax, edgelist=cut, width=6, edge_color=ORANGE)
nx.draw_networkx_nodes(U, pos, ax=ax, nodelist=sorted(S), node_size=900, node_color=BLUE,
edgecolors="white")
nx.draw_networkx_nodes(U, pos, ax=ax, nodelist=sorted(inside), node_size=900, node_color=AQUA,
                       edgecolors="white")
nx.draw_networkx_labels(U, pos, ax=ax, font_size=10, font_color="white", font_weight="bold",
                        labels={n: n[:3] for n in U})
ax.set_title(f"Traffic entering {' and '.join(sorted(inside))}: {need} λ on {links} links\n"
             f"(in orange), hence C ≥ {need}/{links} = {C_min}")
ax.set_axis_off()
ax.margins(0.08)
save(fig, "nx_cut.png")
```

```
strongly connected: True
links that disconnect the network if they fail (bridges): none
node connectivity: 2   edge connectivity: 2
Madrid -> Murcia before the cut: Madrid > Valencia > Murcia
Madrid -> Murcia after the cut : Madrid > Sevilla > Malaga > Murcia
```

```
tightest cut: traffic into ['Malaga', 'Murcia'] = 68 λ over 2 links  ->  every link needs C >= 34
   figure saved: results/networkx_tour/nx_cut.png
```

The last lines of the cell check a cut condition. If the nodes are divided into a set S and the remaining nodes, all the traffic from S to the other nodes has to cross the links that leave S. With the same capacity C on every link this gives

$$C \cdot (\text{number of links leaving S}) \geq \text{traffic from S to the other nodes, for every set S.}$$

The script examines all the 126 possible sets S of Spain-7. The most restrictive one contains all the nodes except Malaga and Murcia: 68 wavelengths have to reach Malaga and Murcia, and only two links enter these nodes (Sevilla to Malaga and Valencia to Murcia). Hence C must be at least 34. The same number appears in handout 3, where the ILP turns out to be infeasible for every capacity below 34.

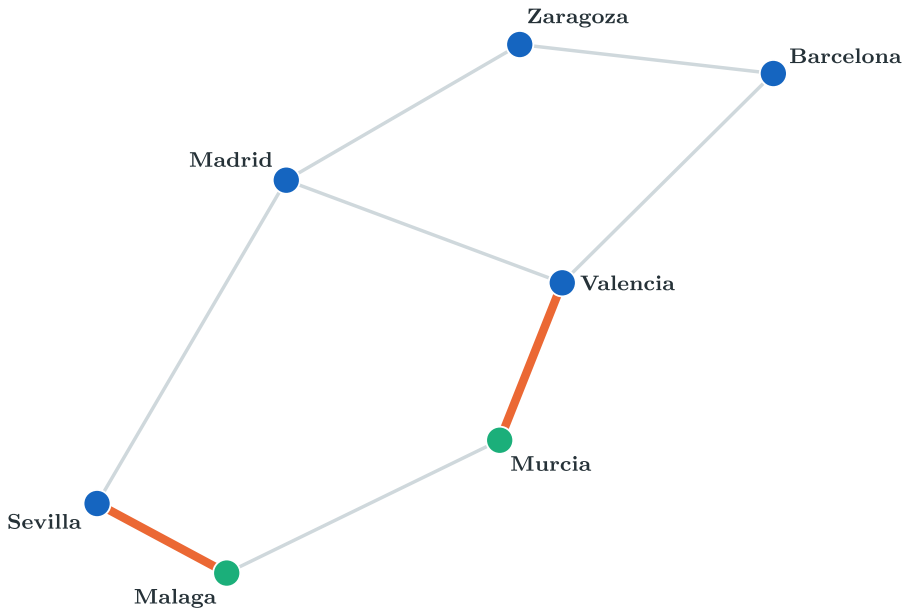


Figure 4. The most restrictive cut for the traffic matrix of the lab: the traffic for Malaga and Murcia (green) must cross the two links in orange.

6. Generating networks

NetworkX contains generators for many families of graphs. The function `pynet2plan.network.from_networkx` converts any of them into a PyNet2Plan network: each undirected edge becomes two links, and the node positions are scaled to a square of 1000 × 1000 units so that the lengths are comparable to kilometres. The result can be saved and opened in the graphical interface.

```
section("6. Generating networks with NetworkX")
generated = {
    "ring: nx.cycle_graph(8)": nx.cycle_graph(8),
    "grid: nx.grid_2d_graph(3, 4)": nx.grid_2d_graph(3, 4),
    "Waxman: nx.waxman_graph(12, seed=7)": nx.waxman_graph(12, beta=0.7, alpha=0.35, seed=7),
    "random: nx.gnm_random_graph(10, 16, seed=3)": nx.gnm_random_graph(10, 16, seed=3),
}
fig, axes = plt.subplots(2, 2, figsize=(11, 9.5))
for ax, (title, Hg) in zip(axes.flat, generated.items()):
    N = network.from_networkx(Hg, capacity=40, name=title.split(":")[0])   # -> a PyNet2Plan network
    p = {n: (a["x"], a["y"]) for n, a in N.nodes(data=True)}
    nx.draw_networkx(N.to_undirected(), p, ax=ax, node_size=160, node_color=BLUE,
edge_color="#546e7a",
                     width=1.5, with_labels=False)
    ok = nx.is_strongly_connected(N)
    kind, call = title.split(": ")
    ax.set_title(f"{kind} - {N.number_of_nodes()} nodes, {N.number_of_edges()} links"
                 f"{'' if ok else ' (NOT connected)'}\n{call}", fontsize=11.5)
    ax.set_axis_off()
    ax.margins(0.08)
    print(f"  {title:45s} {N.number_of_nodes():3d} nodes {N.number_of_edges():3d} links  connected:
{ok}")
save(fig, "nx_generated.png")
W = network.from_networkx(generated["Waxman: nx.waxman_graph(12, seed=7)"], capacity=40, name="Waxman
12")
path = os.path.join(OUT, "waxman12.json")
network.save_topology(W, path)
print(f"  saved {os.path.relpath(path, ROOT)}: open it in the GUI with File > Open topology...")
```

```
  ring: nx.cycle_graph(8)                          8 nodes  16 links  connected: True
  grid: nx.grid_2d_graph(3, 4)                    12 nodes  34 links  connected: True
  Waxman: nx.waxman_graph(12, seed=7)             12 nodes  32 links  connected: True
  random: nx.gnm_random_graph(10, 16, seed=3)     10 nodes  32 links  connected: True
   figure saved: results/networkx_tour/nx_generated.png
  saved results/networkx_tour/waxman12.json: open it in the GUI with File > Open topology...
```

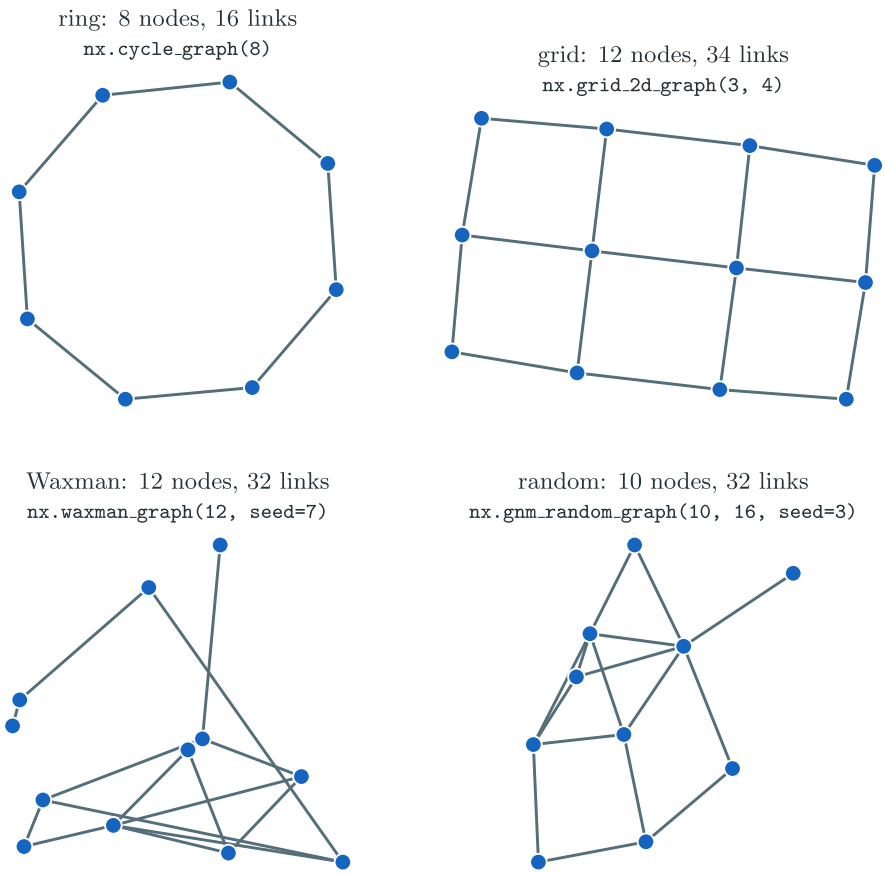


Figure 5. Four generated networks. The Waxman model places the nodes at random and connects nearby nodes with higher probability, as happens in real fibre networks.

7. Repeatable traffic matrices

```
section("7. Traffic: repeatable random matrices")
D1 = traffic.random_traffic(G, 24, 4, 16, seed=9)
D2 = traffic.random_traffic(G, 24, 4, 16, seed=9)
print("same seed, same demands:", D1 == D2)
print("it is exactly data/traffic/spain7_traffic_split_vs_unsplit.csv:", D1 == D)
print("first demands:", ", ".join(f"{d.origin}->{d.destination} {d.traffic}λ" for d in D1[:4]),
"...")
print("total traffic:", traffic.total_traffic(D1), "wavelengths")
print("as a matrix {(origin, destination): wavelengths}:",
dict(list(traffic.traffic_matrix(D1).items())[:3]), "...")
lower = sum(d.traffic * hops[d.origin][d.destination] for d in D)
print(f"if every demand followed a fewest-hops path: {lower} wavelengths used in total"
      f" (a lower bound for the ILP objective)")

if args.show:
    plt.show()
```

```
same seed, same demands: True
it is exactly data/traffic/spain7_traffic_split_vs_unsplit.csv: True
first demands: Sevilla->Barcelona 8λ, Barcelona->Madrid 15λ, Madrid->Sevilla 10λ, Murcia->Valencia 5λ
...
total traffic: 229 wavelengths
```

```
as a matrix {(origin, destination): wavelengths}: {('Sevilla', 'Barcelona'): 8, ('Barcelona',
'Madrid'): 15, ('Madrid', 'Sevilla'): 10} ...
if every demand followed a fewest-hops path: 408 wavelengths used in total (a lower bound for the ILP
objective)
```

The traffic matrix of the lab is therefore the result of `random_traffic(G, 24, 4, 16, seed=9)` and can be regenerated by anybody. The last line computes a lower bound for the ILP of handout 3. If the capacities were unlimited, every demand would follow a path with the fewest hops, and the total number of wavelengths used would be 408.

8. The graph inside an ILP formulation

The formulations in `formulations/` read the network with the same NetworkX calls. As an example, the flow conservation constraints of the splittable flow formulation (handout 3) are written with `G.nodes`, `G.out_edges(n)` and `G.in_edges(n)`:

```
# Flow conservation, for every demand and every node
# ------------------------------------------------------------------
for d in demands:
    for n in G.nodes:
        out_flow = pulp.lpSum(x[d.id, i, j] for (i, j) in G.out_edges(n))
        in_flow = pulp.lpSum(x[d.id, i, j] for (i, j) in G.in_edges(n))
        if n == d.origin:
            rhs = d.traffic
        elif n == d.destination:
            rhs = -d.traffic
        else:
            rhs = 0
        prob += out_flow - in_flow == rhs, f"flow_{d.id}_{n}"
```

Exercises

The exercises are in the file `labs/lab1_exercises.py`. Each one is a function that ends with `raise NotCompletedError(...)`; write the code where the comments indicate and delete that line. The file already contains the network (Spain-7, with 40 wavelengths on every link), the traffic matrix of the lab and three helper functions: `fiber_cut(G, u, v)` returns a copy of the network without the two directions of a link, `fibers(G)` lists the fibres once, and `fewest_hop_bound(G, D)` computes the bound of section 7. The script runs every exercise and reports those not completed yet:

```
python labs/lab1_exercises.py              # all the exercises
python labs/lab1_exercises.py --only 2     # only exercise 2
```

The same exercises are in the notebook `labs/lab1_exercises.ipynb`, which can be opened in Jupyter or VS Code, or in Google Colab without installing anything (*File > Upload notebook*, or the link *Open in Colab* at the top of the notebook in the lab repository). Its first part is a short introduction to NetworkX that follows sections 1 to 7 of this handout, and its second part contains the exercises. The notebook is self-contained: it includes the networks and traffic matrices used by the exercises, the few functions of PyNet2Plan that they call, and a function `draw(H, paths)` that shows a network and some paths on a map. Changes made in Colab are kept only with *File > Save a copy in Drive*.

exercise	question	NetworkX
1	Paths from Sevilla to Valencia with the fewest hops and in kilometres, before and after the cut of the fibre Madrid-Valencia. How many hops and kilometres does the cut add?	`shortest_path`, `path_weight`
2	For each of the 8 single fibre cuts: does the network remain connected, and how many wavelengths does the fewest-hop routing of the lab traffic need? Which cut is the most harmful?	`is_strongly_connected`, `shortest_path_length`
3	The cut bound of section 5 for the other traffic matrix, `data/traffic/spain7_traffic.csv`: which set S is the most restrictive, and which capacity does it require?	`itertools.combinations`
4	With the fewest-hop routing of the lab traffic, how many wavelengths cross each node in transit (neither origin nor destination)?	`shortest_path`
5	Add Bilbao, linked to Madrid and Zaragoza with 40 wavelengths, generate 30 demands of 4 to 16 wavelengths with seed 3, and save both files. Open them in the graphical interface and run the splittable flow formulation. Is the problem feasible? Find the smallest feasible capacity with a sweep and compare it with the cut bound of exercise 3.	`add_city`, `add_link`, `random_traffic`
6 (optional)	Generate the first connected Waxman network with 15 nodes (section 6) and open it in the graphical interface.	`waxman_graph`, `from_networkx`
7 (optional)	Number of nodes and links, links per node and diameter in hops of Spain-7, NSFNET-14, COST239 and GEANT2.	`diameter`

Compare the results of exercise 2 with the fibre cuts of handout 3 (section 5), where the capacity of the links is limited to 60 wavelengths.